

Computer Science & IT

Theory of Computation



Turing Machines

Lecture No. 3



By- Ravindra Sir

Recap of Previous Lecture



Topic

TM

Topic

Topic

Topics to be Covered



Topic

T_m

Topic

Reduction theorem

Topic

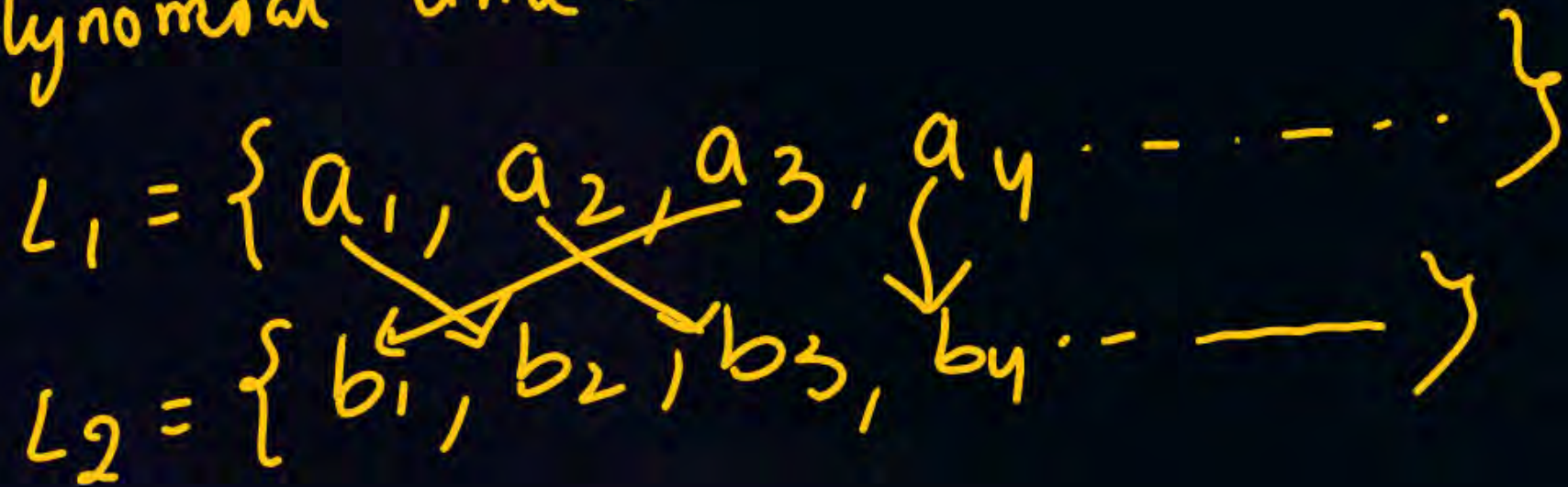
Reduction theorem

$$L_1 \leq L_2$$

L_1 is reducible to L_2 .

It means every string in L_1 can be reduced to a string in L_2 in polynomial time.

$$L_1 = \{a_1, a_2, a_3, a_4, \dots\}$$

$$L_2 = \{b_1, b_2, b_3, b_4, \dots\}$$


$$L_1 \leq L_2$$

$$UD \rightarrow UD$$

$$D \leftarrow D$$

$$D \rightarrow \text{no Comments}$$

$$\text{no Comments} \leftarrow UD$$

$$Rec \leftarrow Rec$$

$$SD \leftarrow SD$$

$$RE \leftarrow RE$$

$$\text{Not Rec} \rightarrow \text{Not Rec}$$

$$\text{Not SD} \rightarrow \text{Not SD}$$

$$\text{Not RE} \rightarrow \text{Not RE}$$

If $L_1 \leq L_2$, L_1 is UD, then $L_2 = ?$ UD

If $L_1 \leq L_2$, L_1 is D, then $L_2 = ?$ No Comments
 $\rightarrow$ may be Decidable
 $\quad \quad \quad \uparrow$
 $\quad \quad \quad$ undecidable

Gate:-

→ $L_1 \leq L_2$ and $L_2 \leq L_3$, L_2 is RE, then $L_1 = ?$, $L_3 = ?$.

$\downarrow$
RE ← RE

L_1 is RE

RE

$\downarrow$

no
Comments

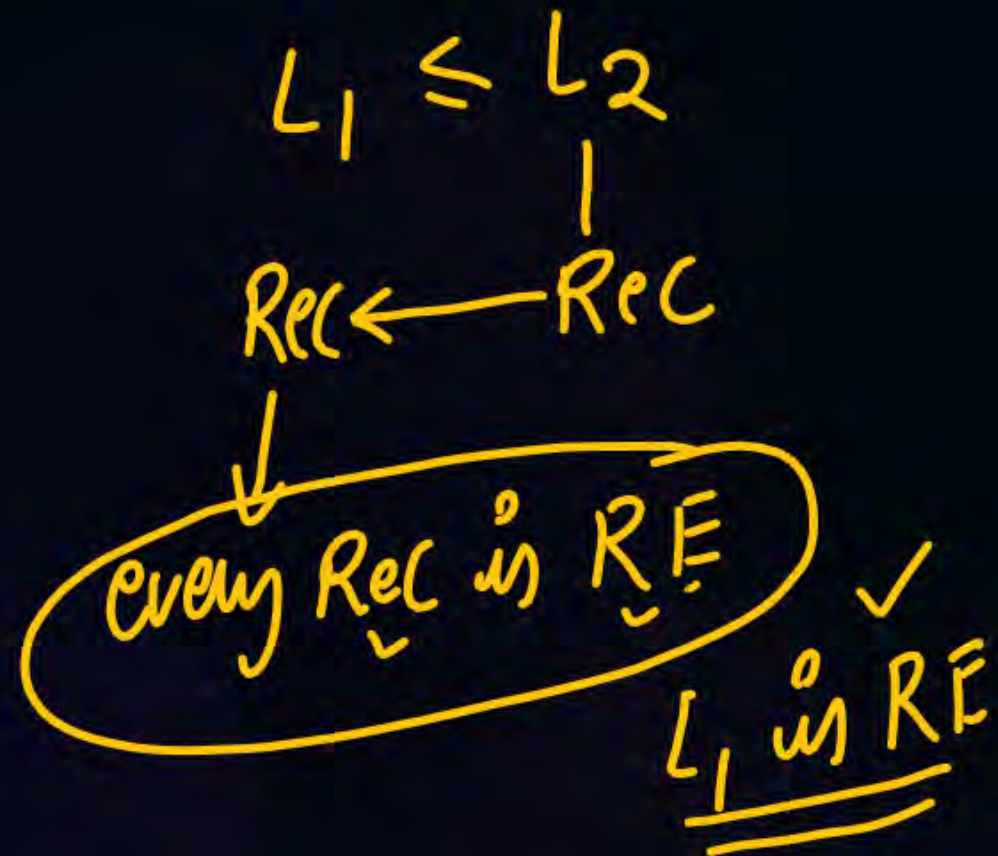
$$\begin{array}{ccc}
 L_1 \leq L_2 & \text{and} & L_2 \leq L_3 \\
 \downarrow & & \downarrow \\
 \text{UD} \rightarrow \text{UP} & & \text{UD} \rightarrow \text{UP}
 \end{array}$$

$$L_1 \leq L_2 \quad \text{and} \quad L_2 \leq L_3$$

$$\text{Dec} \leftarrow \text{Dec} \qquad \text{Dec} \leftarrow \text{Dec}$$

If $L_1 \leq L_2$ and L_2 is Rec, then L_1 is RE. (T/F)

a) True b) False

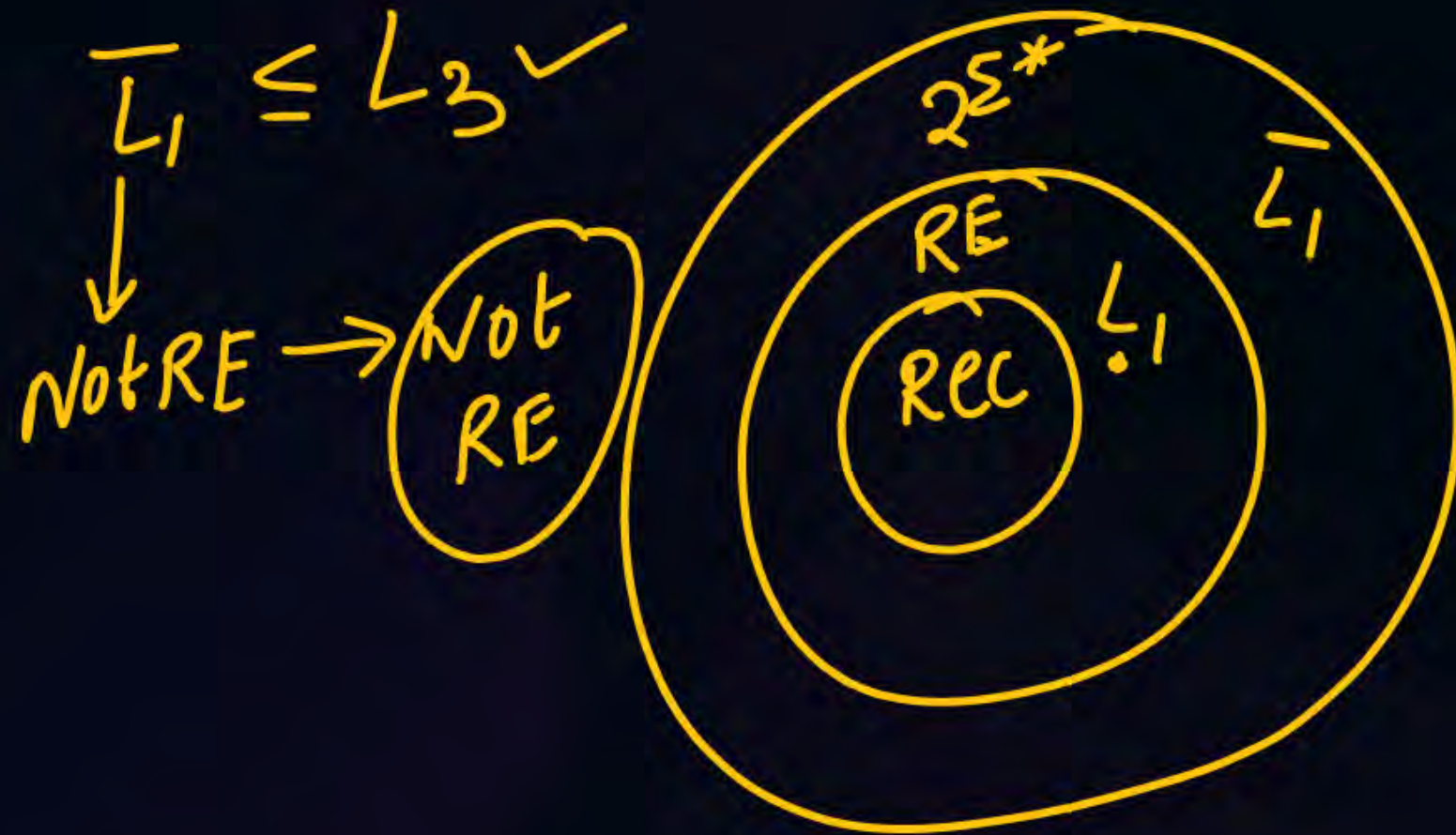


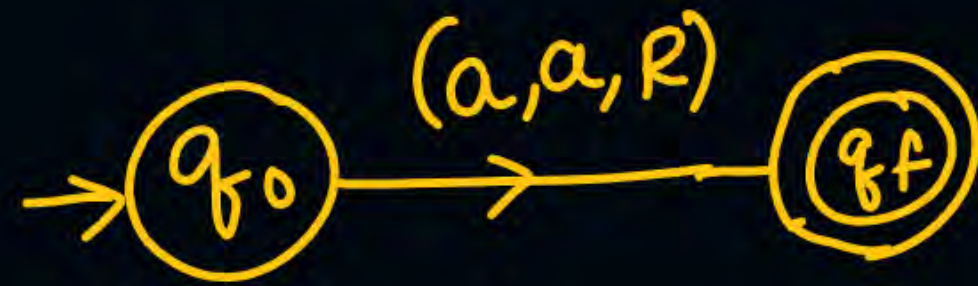
$$\begin{array}{c}
 L_1 \cup L_2 \leq L_3 \\
 \downarrow \quad \downarrow \\
 RE \uparrow \underline{REC} \cup RE \\
 \underbrace{\quad \quad \quad}_{\leftarrow RE \rightarrow X} \\
 L_3 = ? \\
 \text{No comments}
 \end{array}$$

$$\begin{array}{c}
 L_3 \leq L_1 U L_2 \\
 \uparrow \text{REC} \quad \text{RE} \\
 \text{RE} \leftarrow \underbrace{\text{RE} U \text{RE}}_{\text{RE}}
 \end{array}$$

$$\overline{L_1} \leq L_3$$

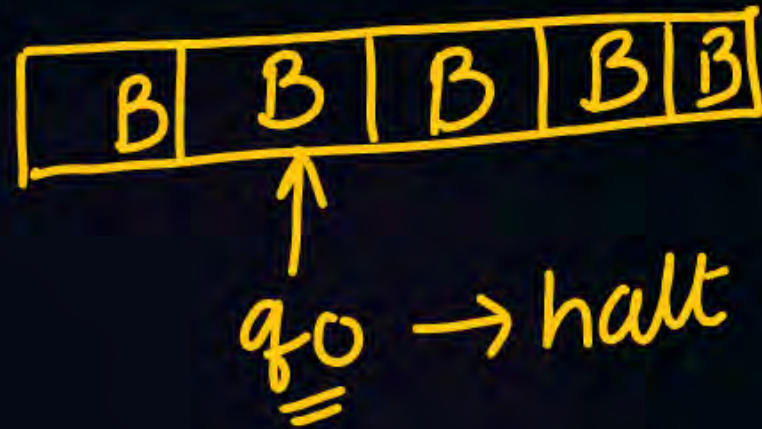
L_1 is RE but not REC

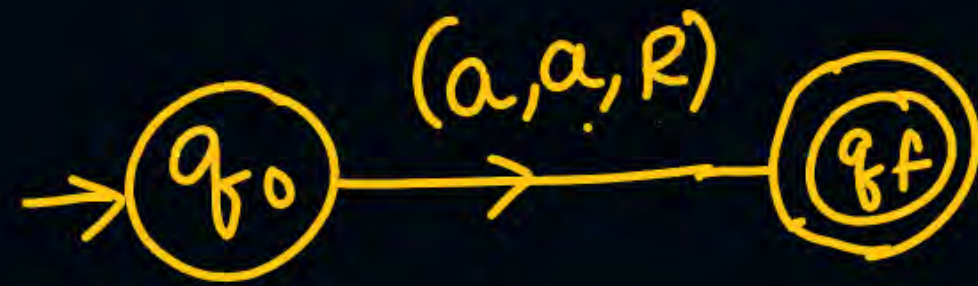




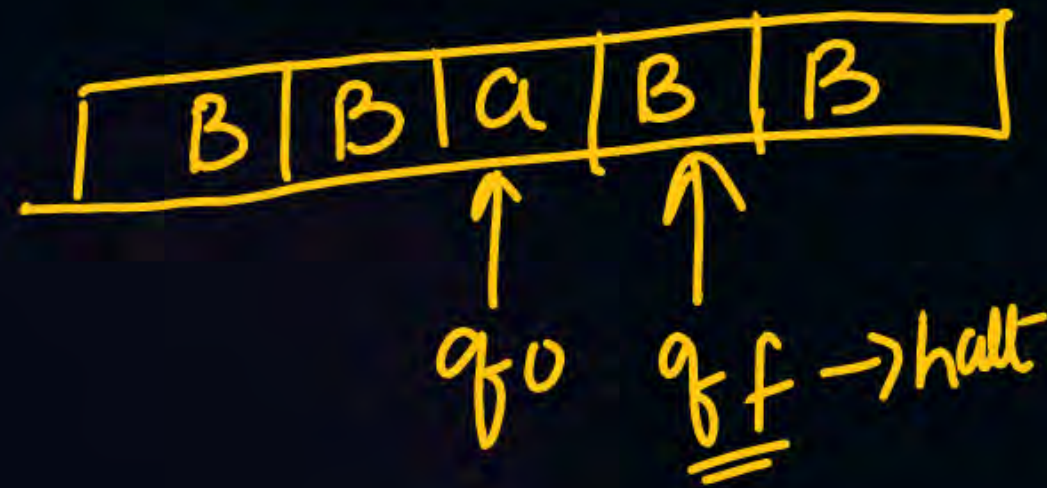
$\epsilon \rightarrow \text{halt, Reject}$

ϵ ✓
 a
 b
 aa
 ab
 aaa

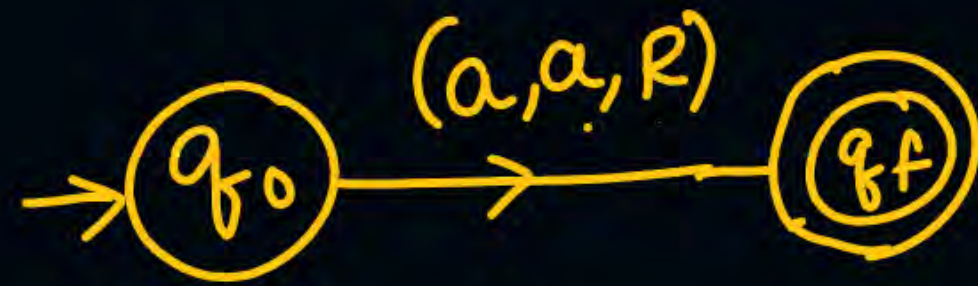




ϵ ✓
 a ✓
 b
 aa
 ab
 aaa



$\epsilon \rightarrow$ halt, Reject
 $a \rightarrow$ halt, accept

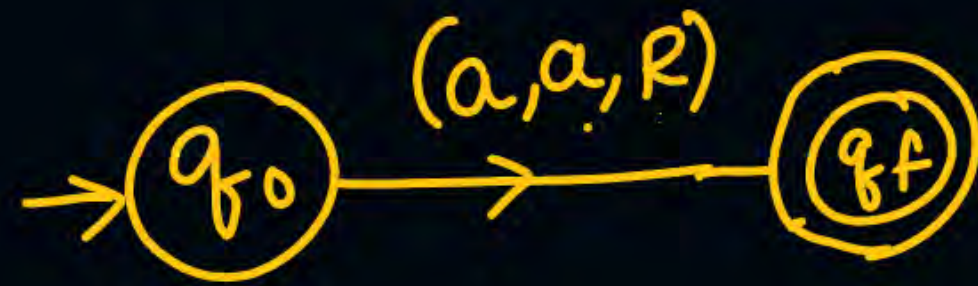


ϵ ✓
 a ✓
 b ✓
 aa
 ab
 aaa

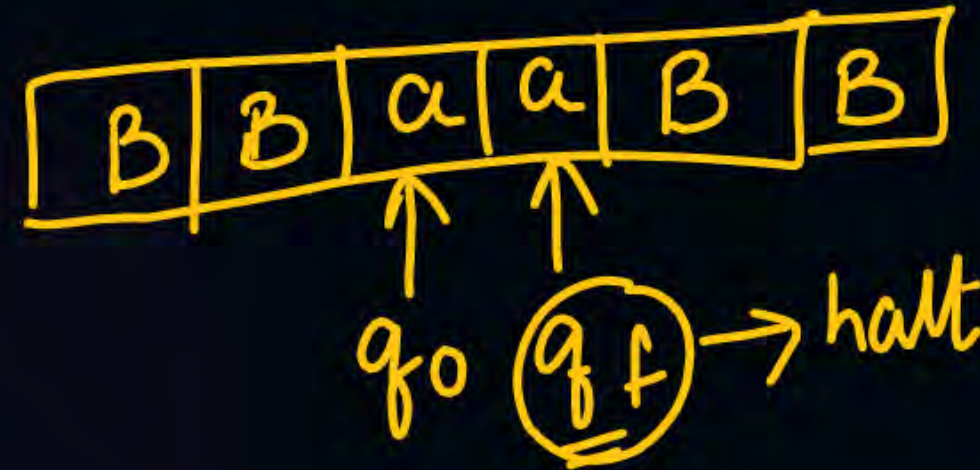


$q_0 \rightarrow \underline{\underline{\text{halt}}}$

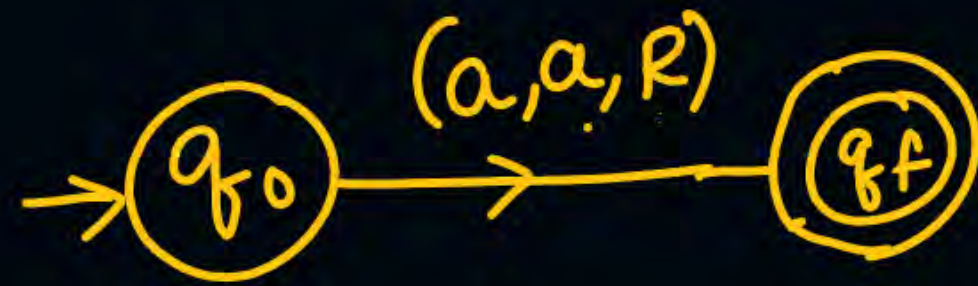
$\epsilon \rightarrow \text{halt, Reject}$
 $a \rightarrow \text{halt, accept}$
 $b \rightarrow \text{halt, Rej}$



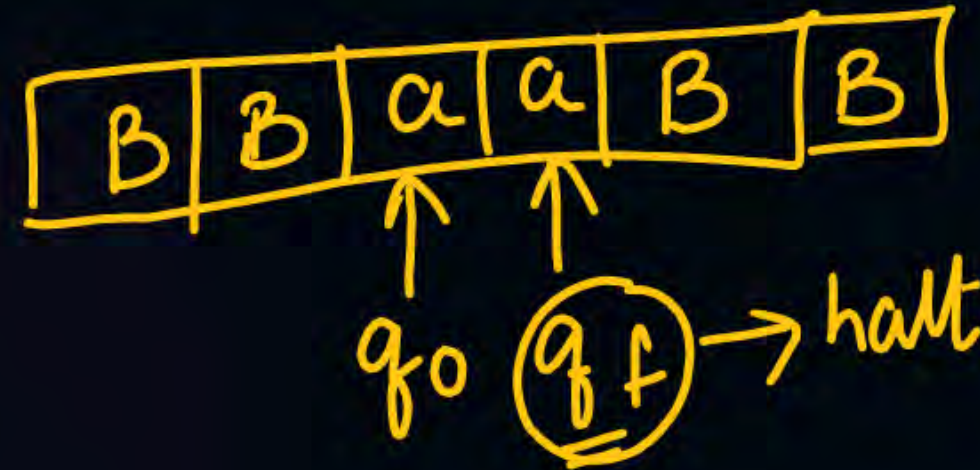
ϵ ✓
 a ✓
 b ✓
 aa ✓
 ab
 aaa



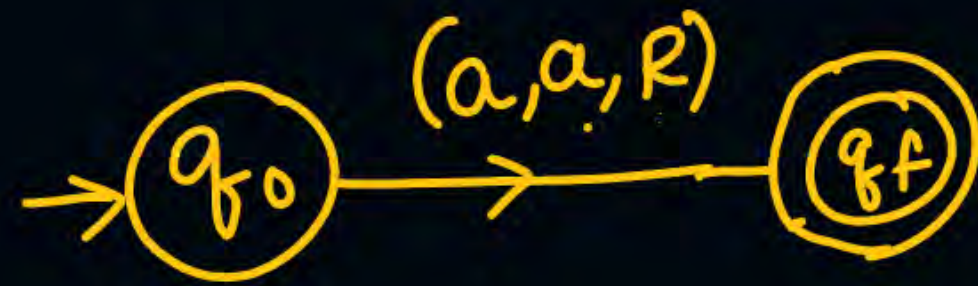
$\epsilon \rightarrow \text{halt, Reject}$
 $a \rightarrow \text{halt, accept}$
 $b \rightarrow \text{halt, Rej}$
 $aa \rightarrow \text{halt, acc}$



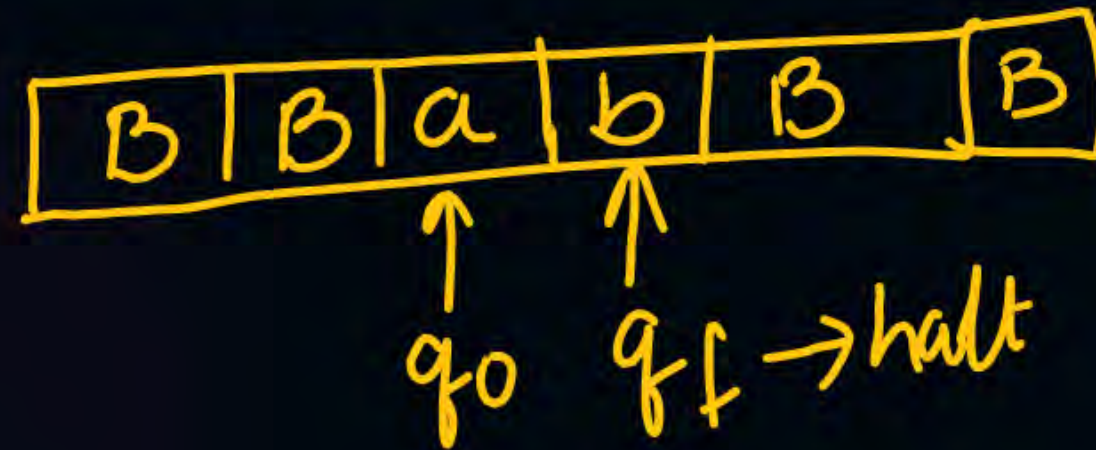
ϵ ✓
 a ✓
 b ✓
 aa ✓
 ab
 aaa



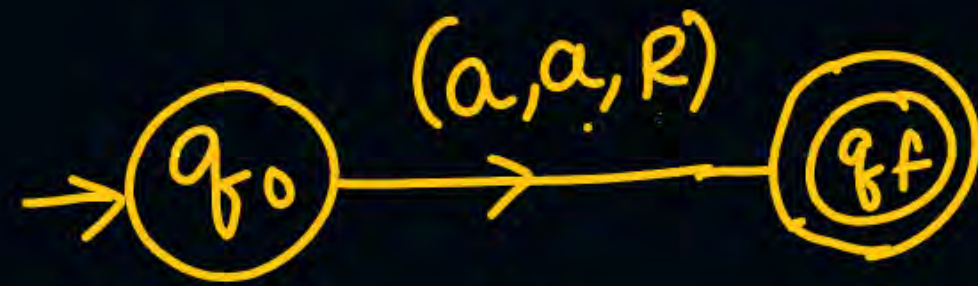
$\epsilon \rightarrow \text{halt, Reject}$
 $a \rightarrow \text{halt, accept}$
 $b \rightarrow \text{halt, Rej}$
 $aa \rightarrow \text{halt, acc}$



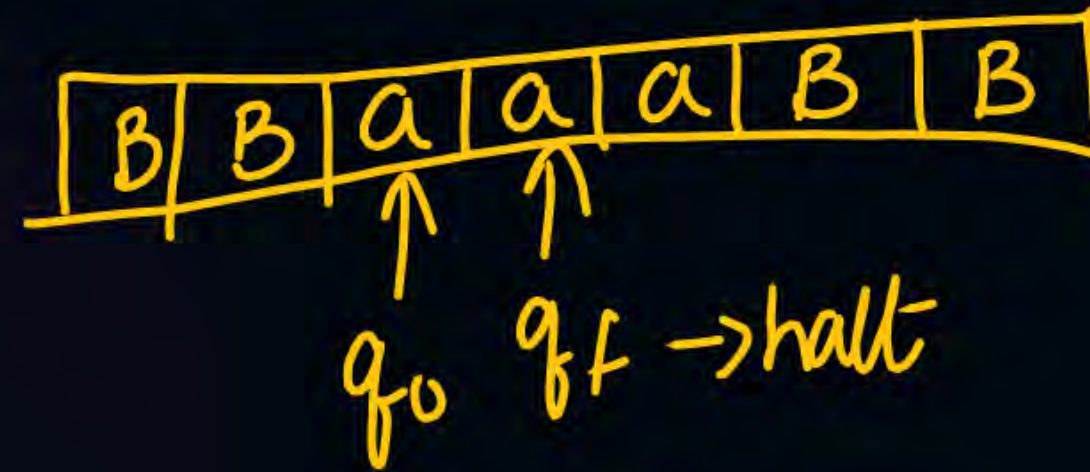
ϵ ✓
 a ✓
 b ✓
 aa ✓
 ab
 aaa



$\epsilon \rightarrow \text{halt, Reject}$
 $a \rightarrow \text{halt, accept}$
 $b \rightarrow \text{halt, Rej}$
 $aa \rightarrow \text{halt, acc}$
 $ab \rightarrow \text{halt, acc}$



ϵ ✓
 a ✓
 b ✓
 aa ✓
 ab
 aaa

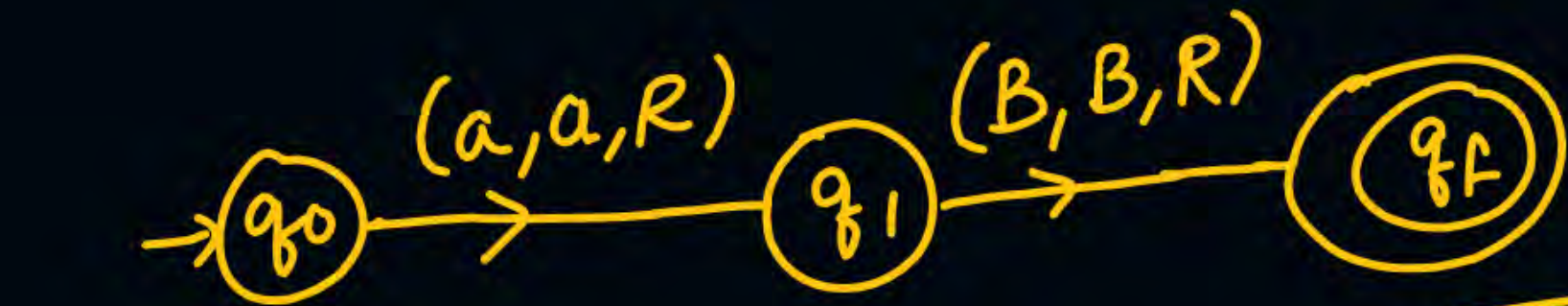


$a(a+b)^*$ ✓

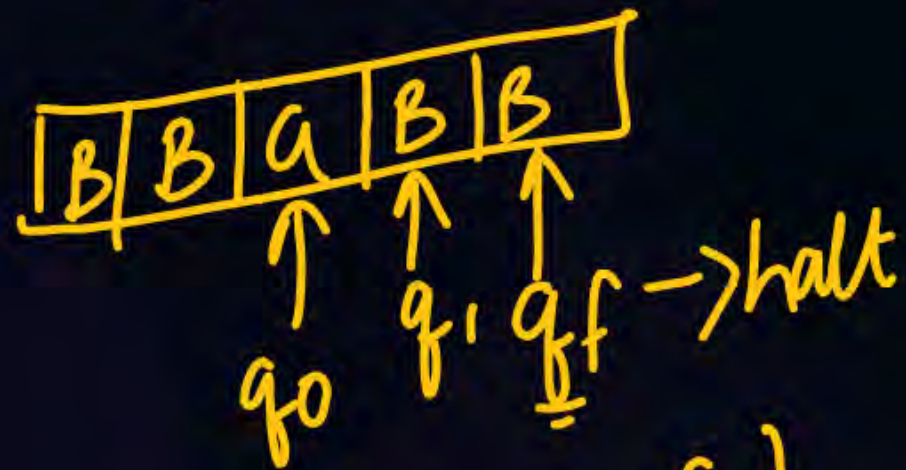
$\epsilon \rightarrow \text{halt, Reject}$
 $a \rightarrow \text{halt, accept}$
 $b \rightarrow \text{halt, Rej}$
 $aa \rightarrow \text{halt, acc}$
 $ab \rightarrow \text{halt, acc}$
 $aaa \rightarrow \text{halt acc}$

If TM only moves R & only moves L, then Language
Accepted is regular language

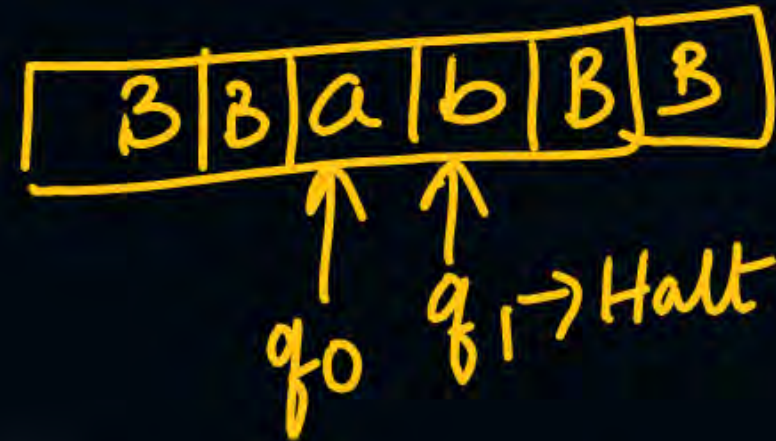
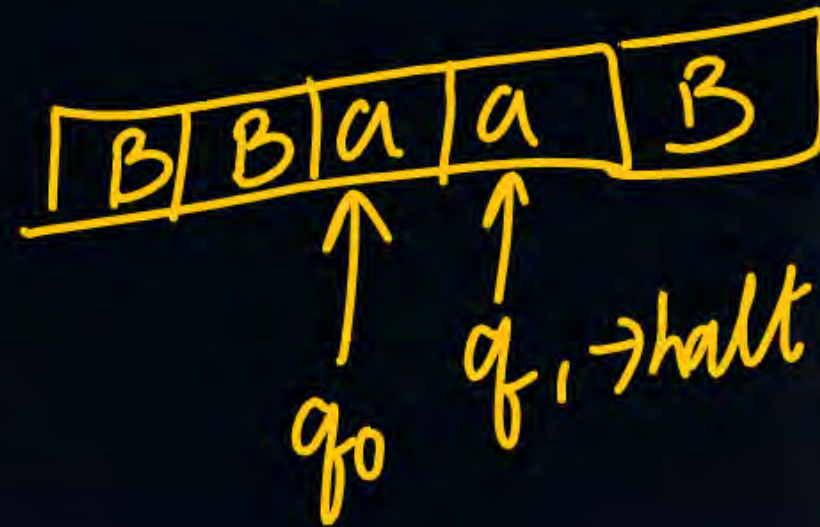
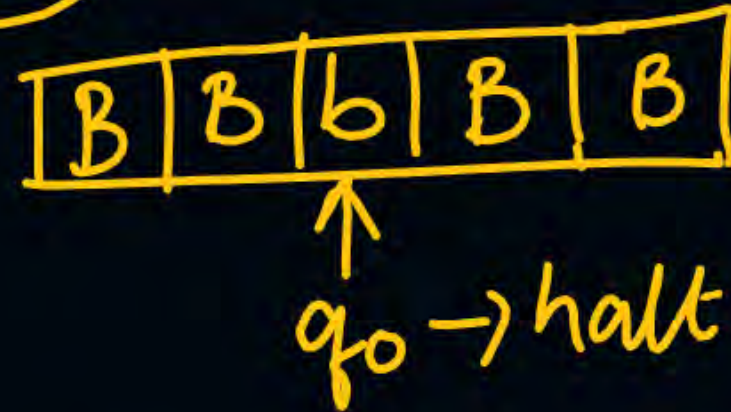




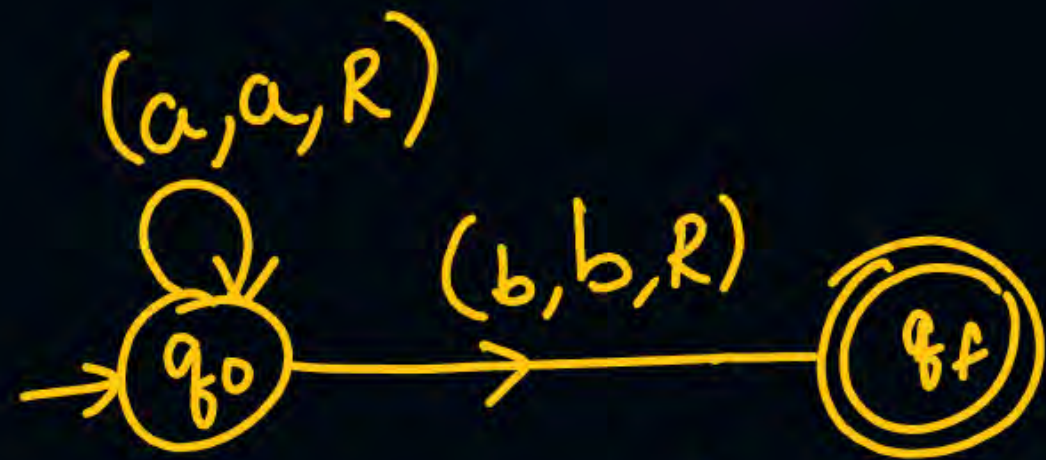
$\in X$
a ✓
 b ✗
 a a ✗
 a b ✗



Language = {a}

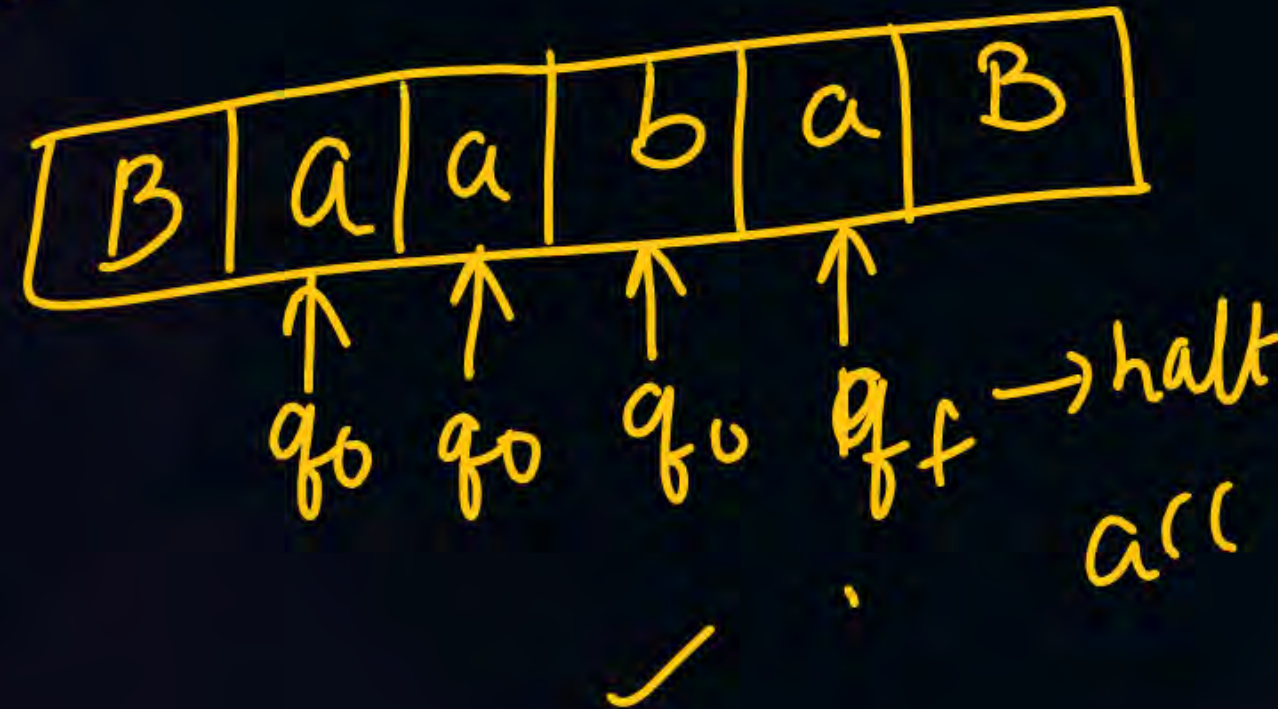


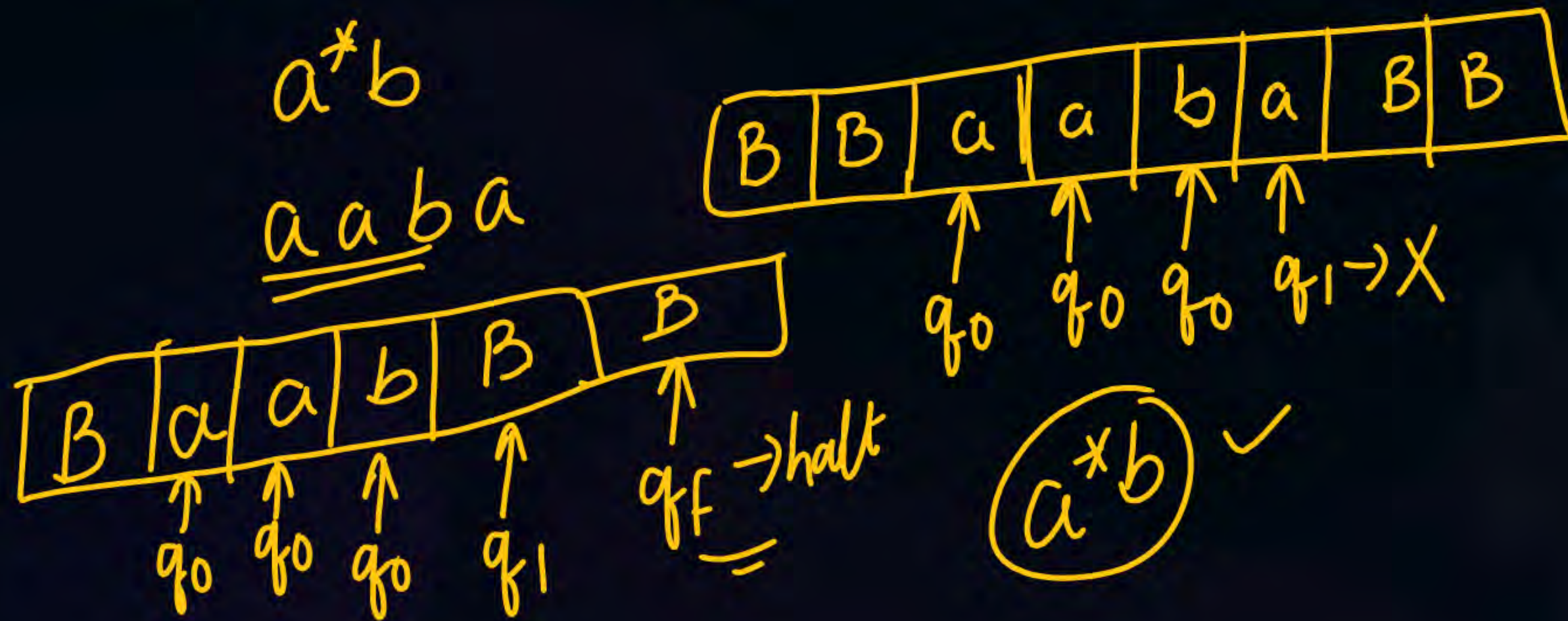
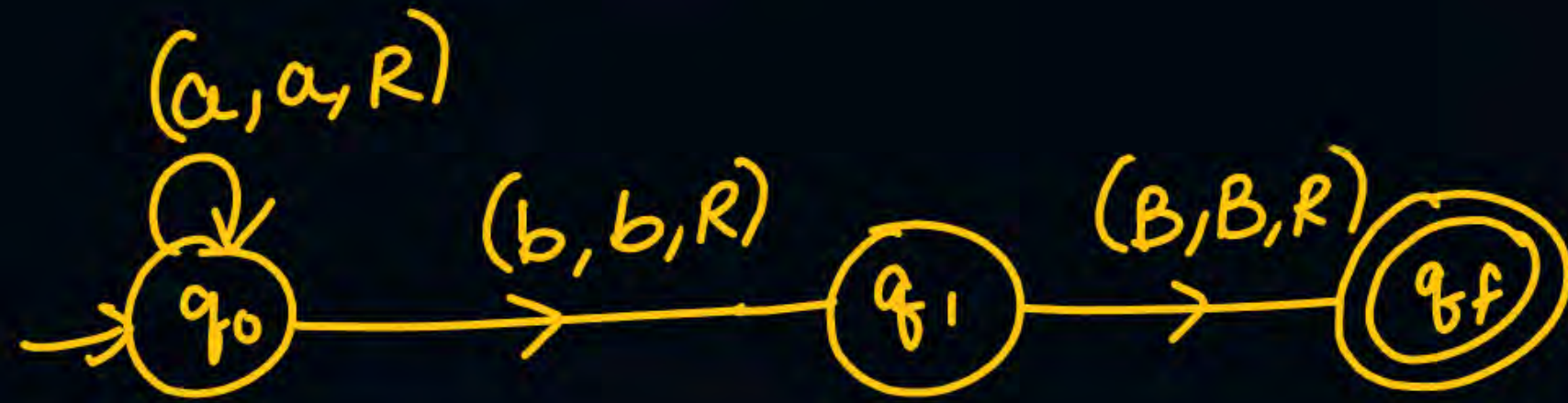
halts on $(a+b)^*$ ✓

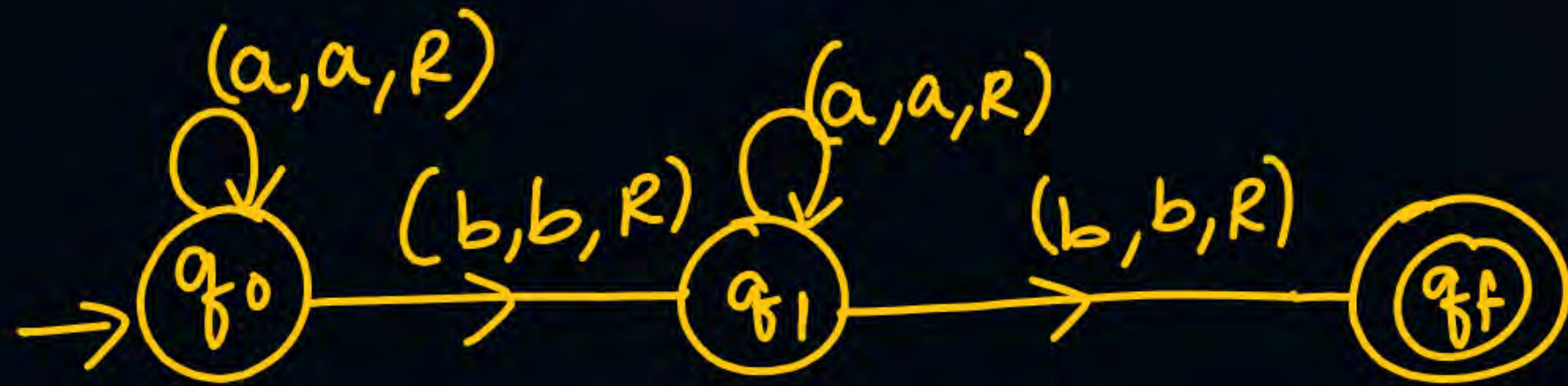


$a^*b(a+b)^*$ ✓

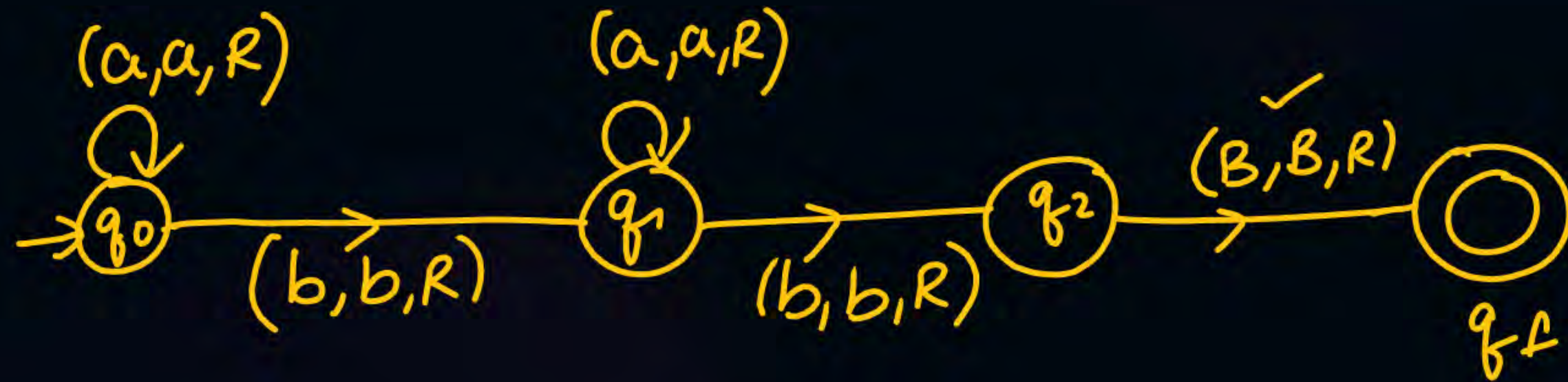
aaba



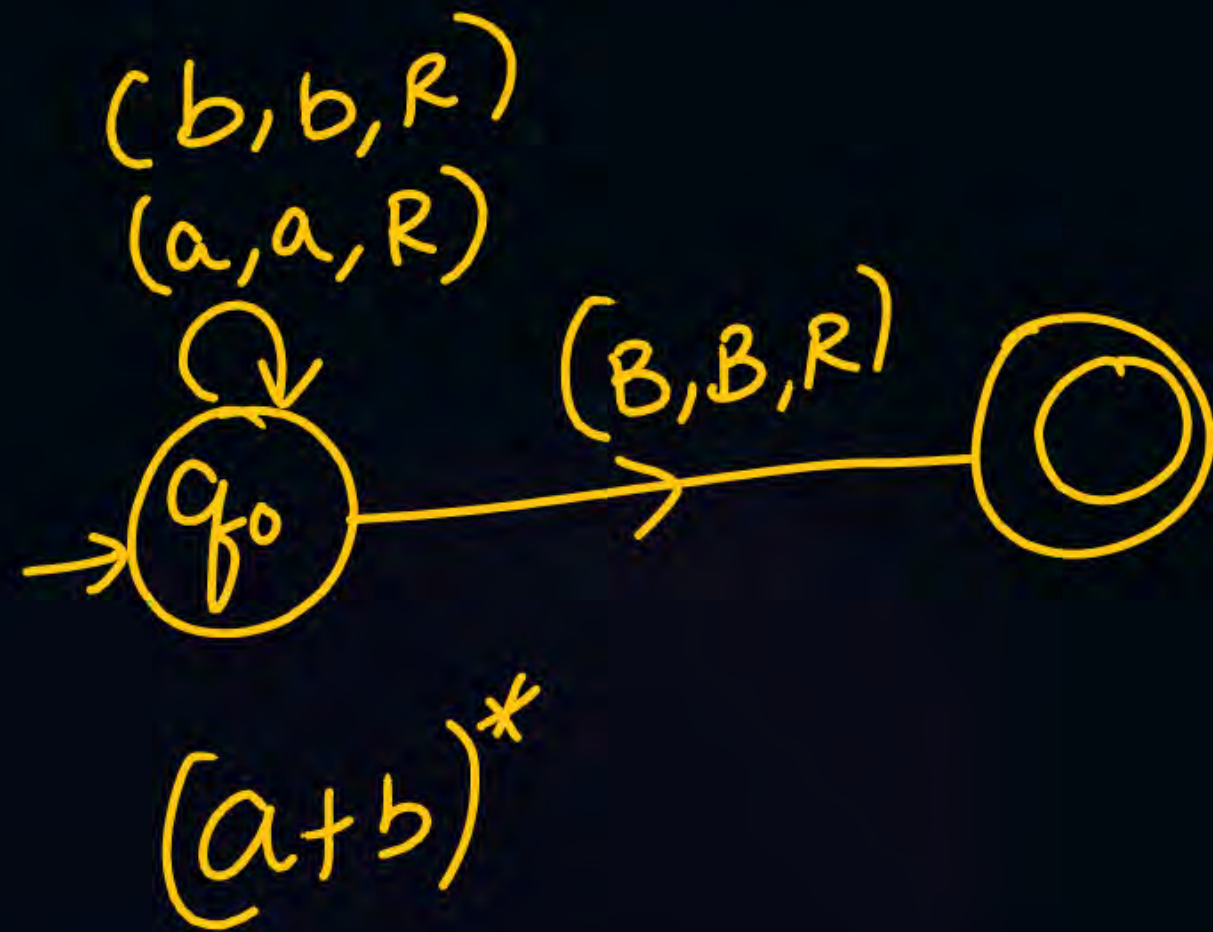


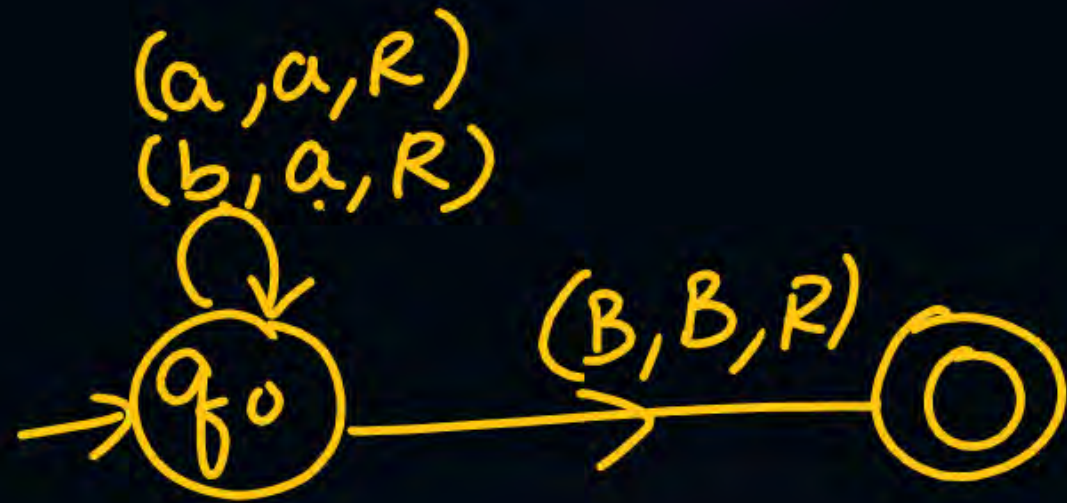


$a^* b a^* b \underline{(a+b)^*}$ ✓



$a^* b a^* b$

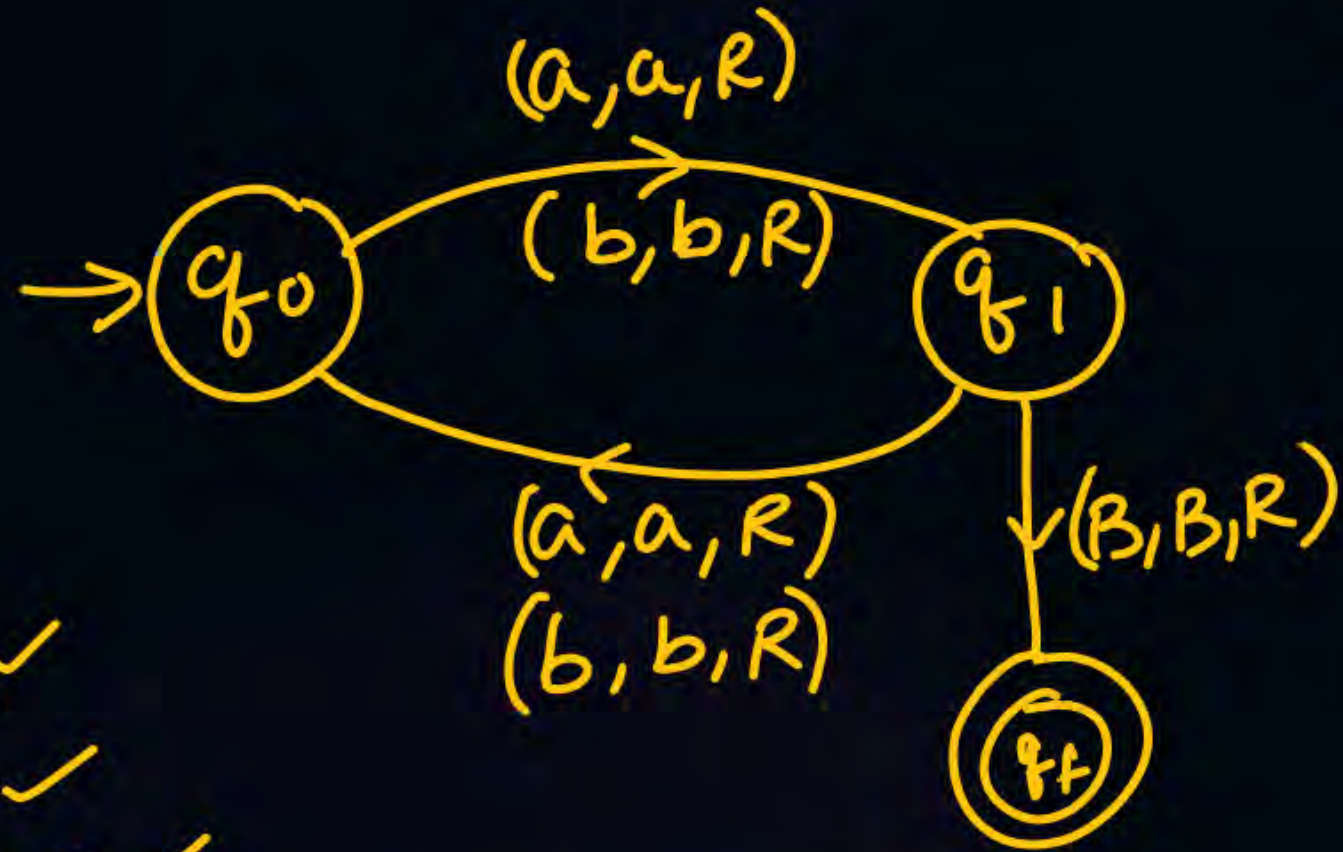




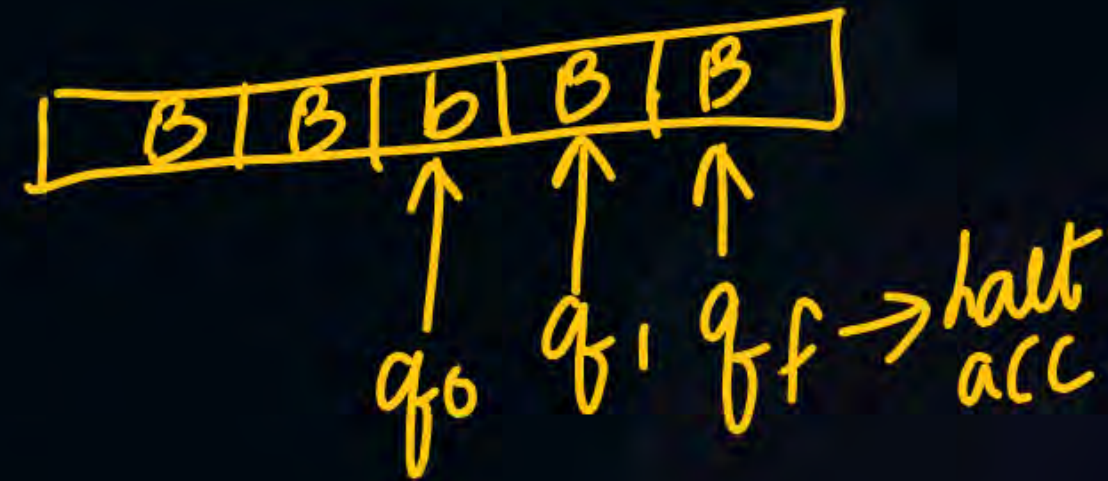
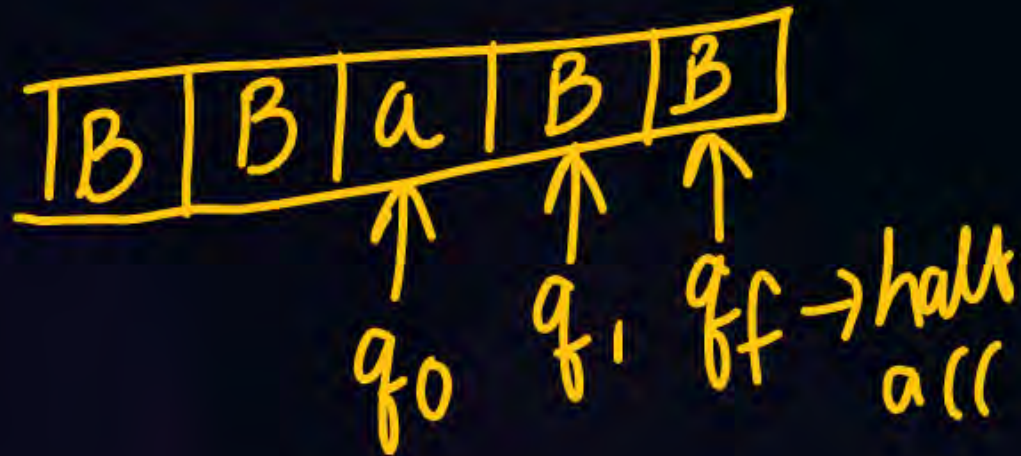
$(a+b)^*$

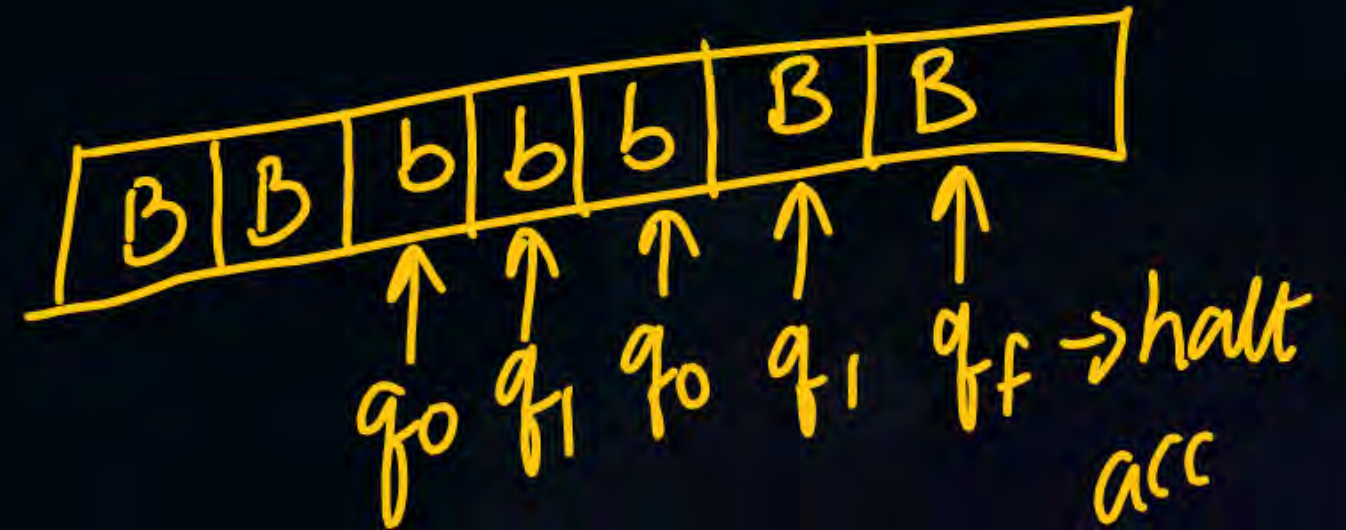
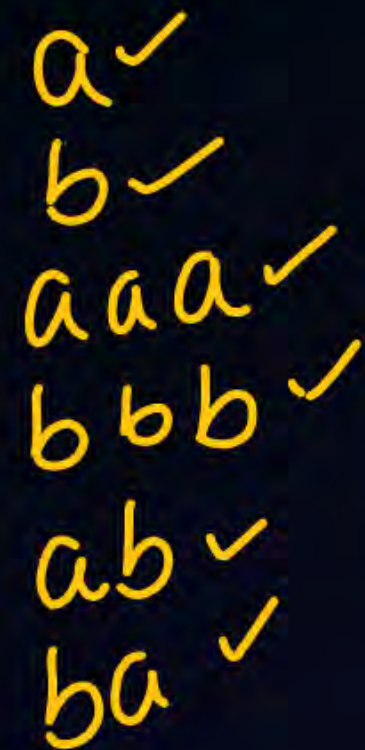
Turning all the symbols on tape to 'a'

5 min

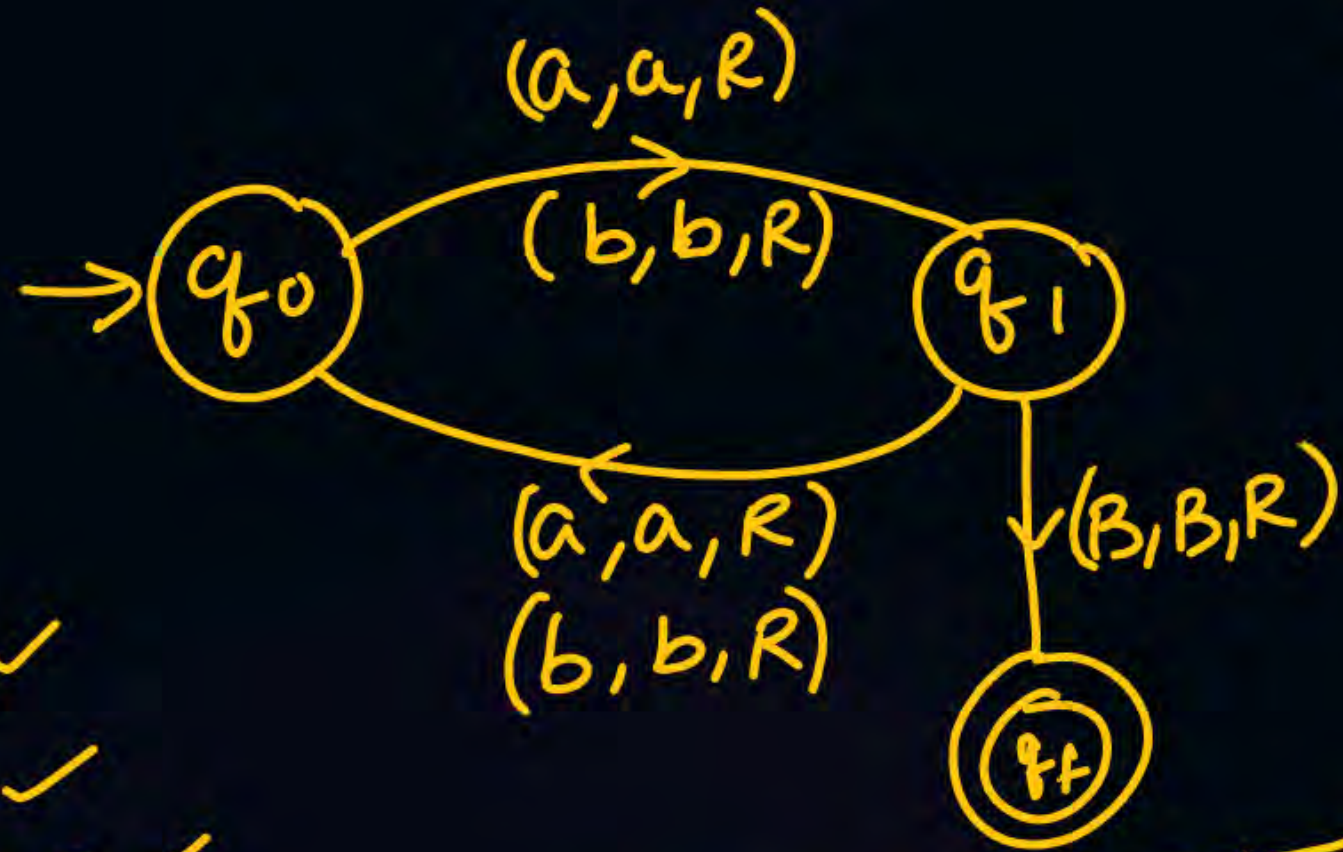


a✓
 b✓
 aaa✓
 bbb✓
 ab
 ba



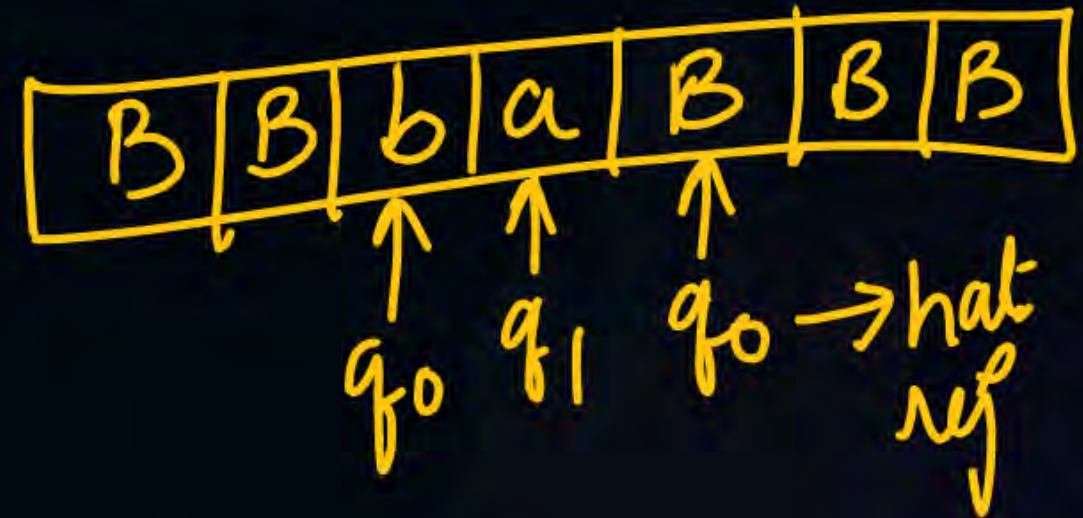
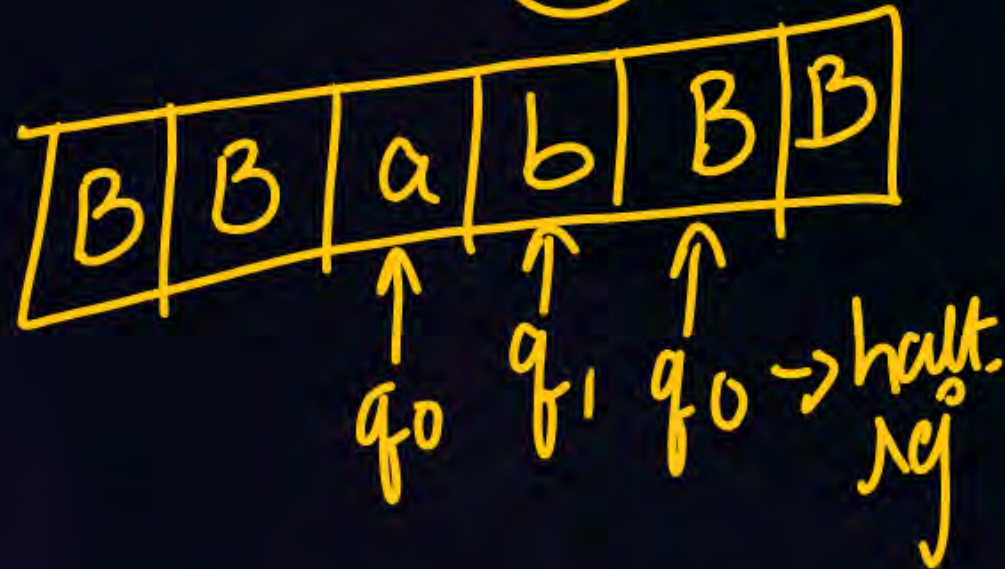


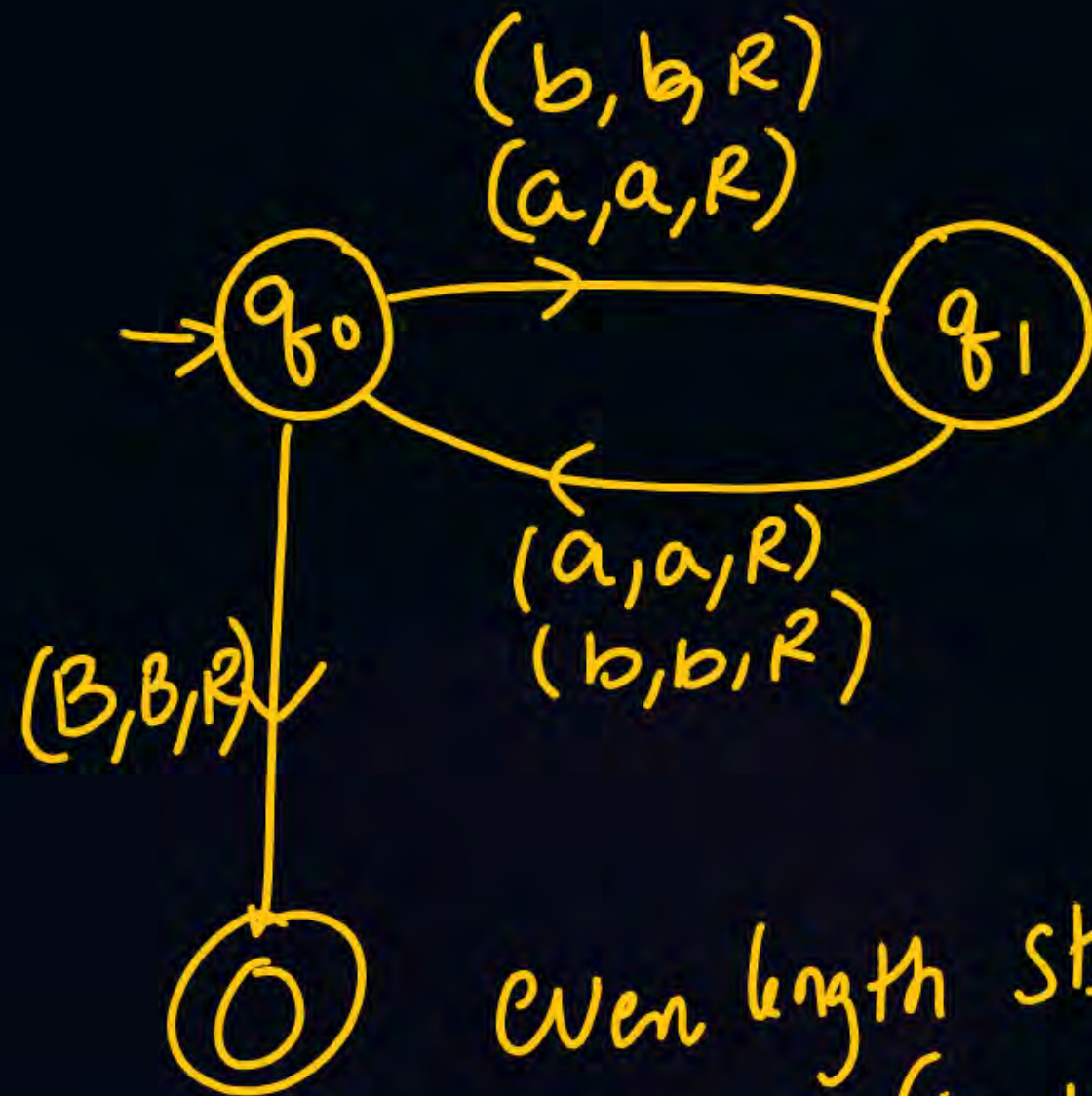
5 min



$$(a+b)(a+b)^*(a+b)$$

a ✓
 b ✓
 aaa ✓
 bbb ✓
 ab ✓ x
 ba ✓ x

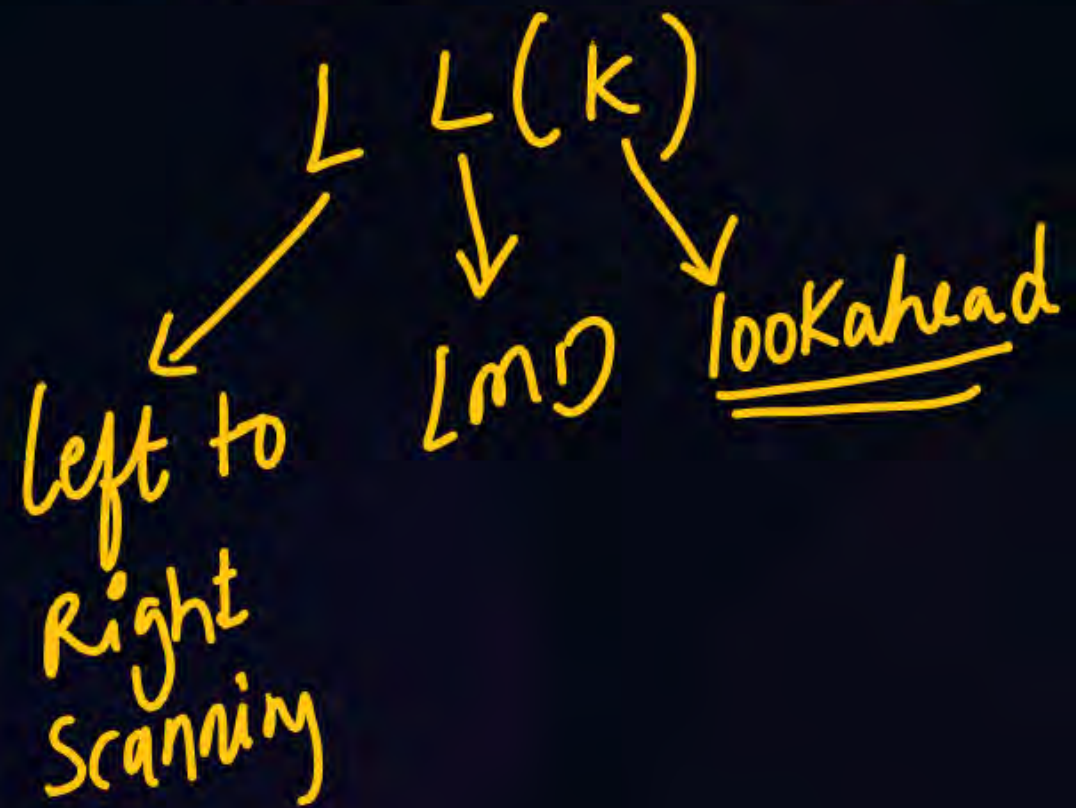




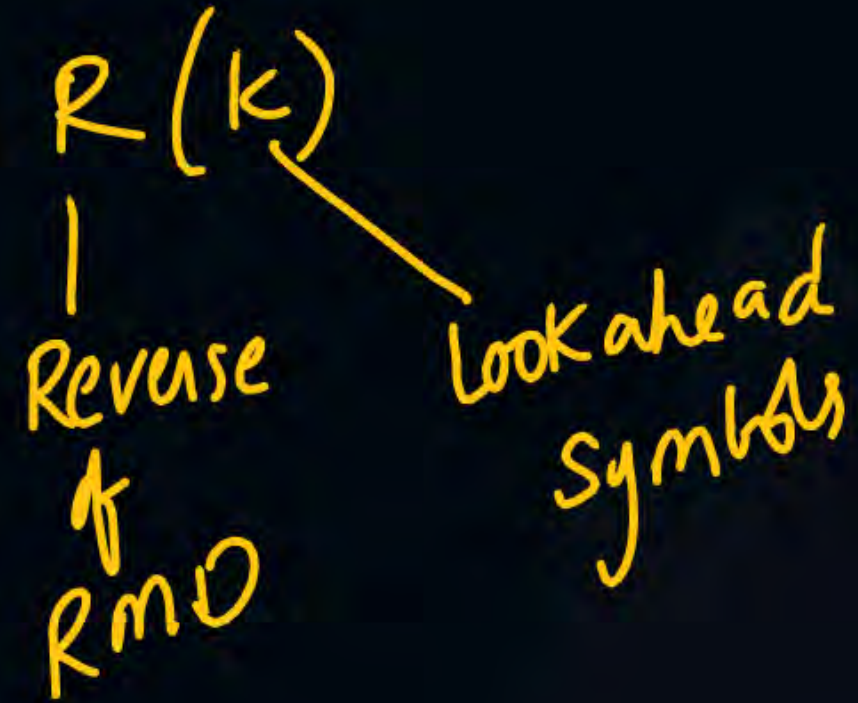
even length string
 $((a+b)(a+b))^*$

membership for CFL :- $w \in L$?

- 1) Brute force X
- 2) CYK $\rightarrow$ Parser $\checkmark$
- 3) LL(k) LR(k) $\rightarrow$ parser



L
left to Right

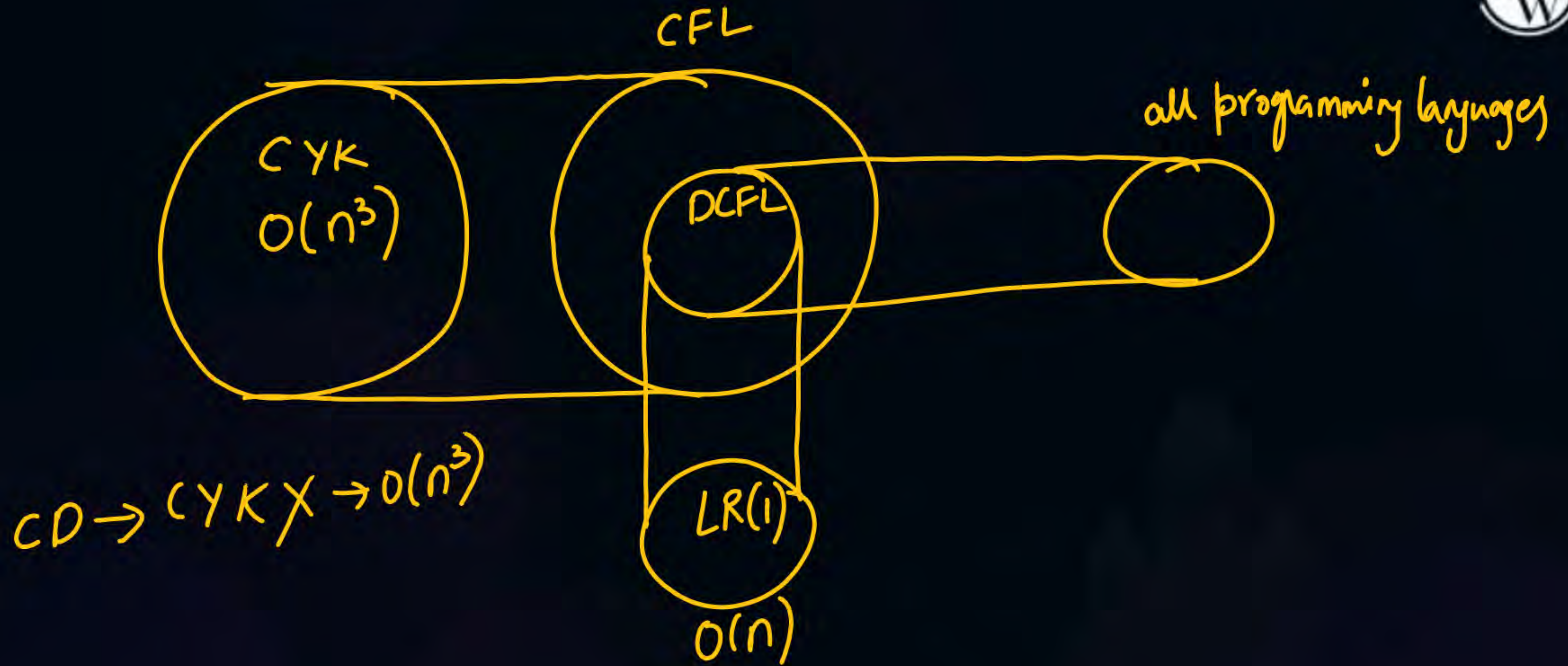


Properties of $LL(k)$ and $LR(k)$:-

- 1) $LL(k)$ and $LR(k)$ grammars are unambiguous
- 2) Some unambiguous grammars are neither

$LL(k)$ nor $LR(k)$

- 3) If a grammar is ambiguous, then surely it is not $LL(k)$ or $LR(k)$



LL(k):- A grammar is LL(k), when k symbols of I/P
✓ are presented to compiler at a time, the production that
is used in every step of derivation is uniquely determined

CD

$$LL(k) \rightarrow TC \rightarrow O(n)$$



LR(k):- A grammar is LR(k) if when k symbols of I/P are presented to compiler at a time, the production that is used in every step of derivation is uniquely determined

$$LR(k) \rightarrow TC \rightarrow O(n)$$

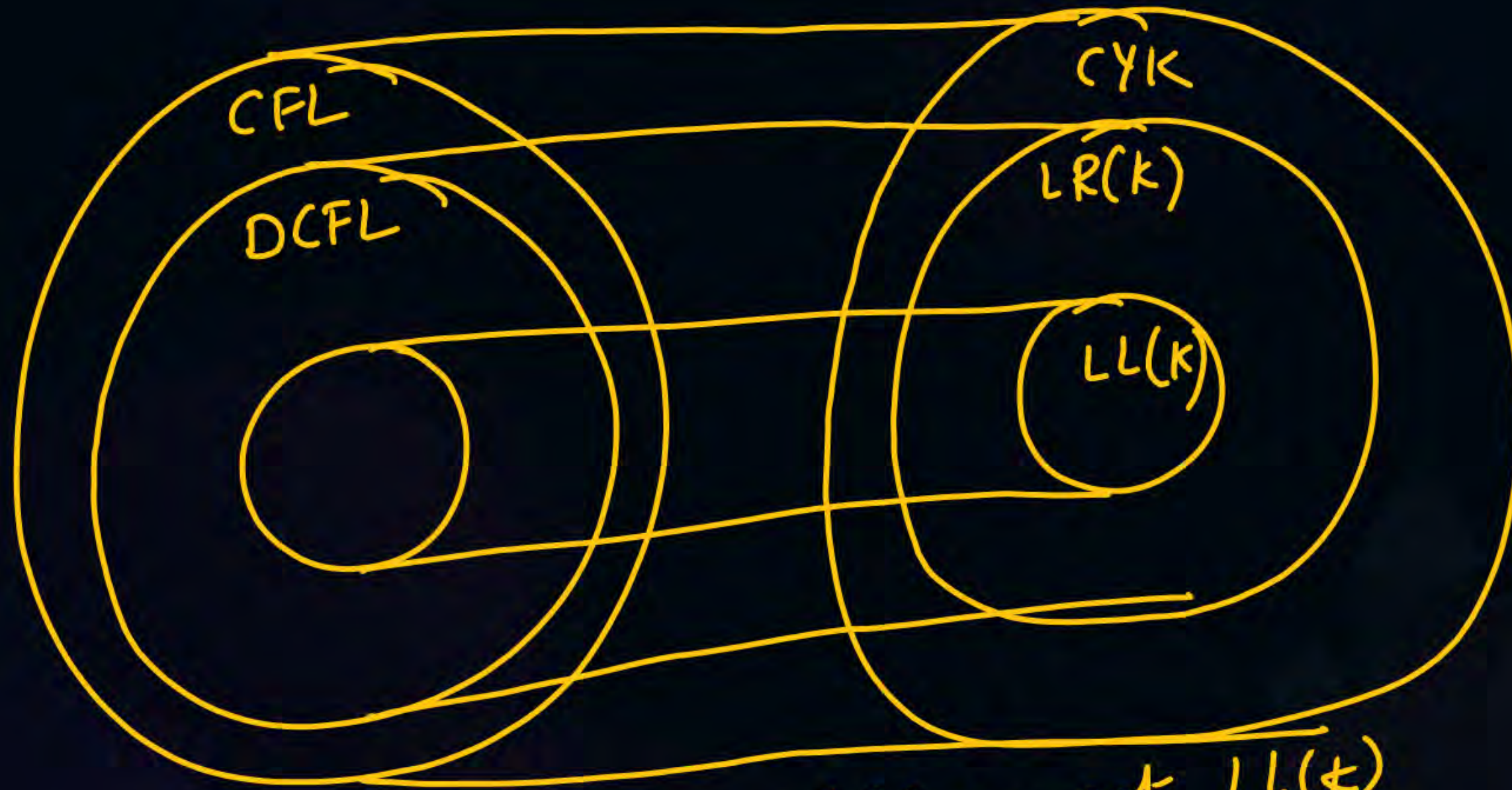
$S \rightarrow \underline{a} / \underline{a}b$ $\Rightarrow$ non deterministic
XLL
 $\downarrow$
 $\textcircled{CP}$ ✓

$\forall \text{ DCFL} \rightarrow \exists \text{ LR}(k) / k \geq 1$



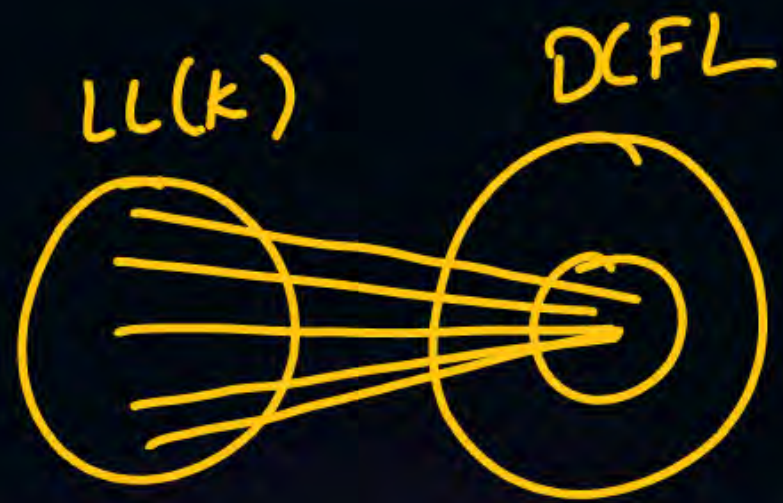
$\nexists \text{ LR}(k) / k \geq 1 \rightarrow \exists \text{ DCFL}$





There are some DCFLS which are not $LL(k)$

- In $LL(k)$ and $LR(k)$ every string will have exactly one derivation tree. Because DCFL are unambiguous
- Every feature of any programming language can be represented using DCFL. So we don't use CYK for parsing



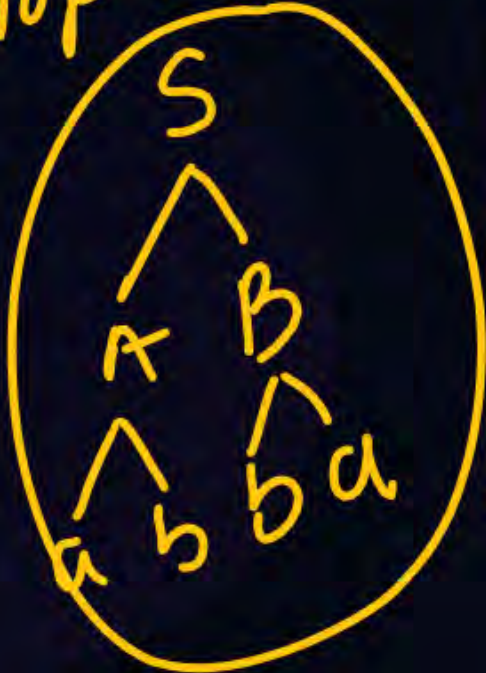
Every $DCFL$ is not $LL(k)$

→ If a grammar is $LL(k)$, we can S

$LL(k)$



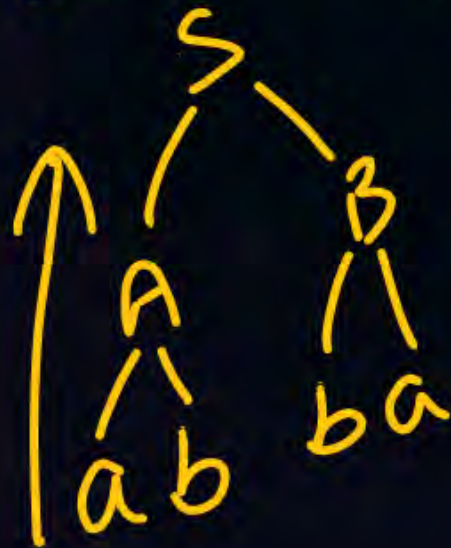
Top down



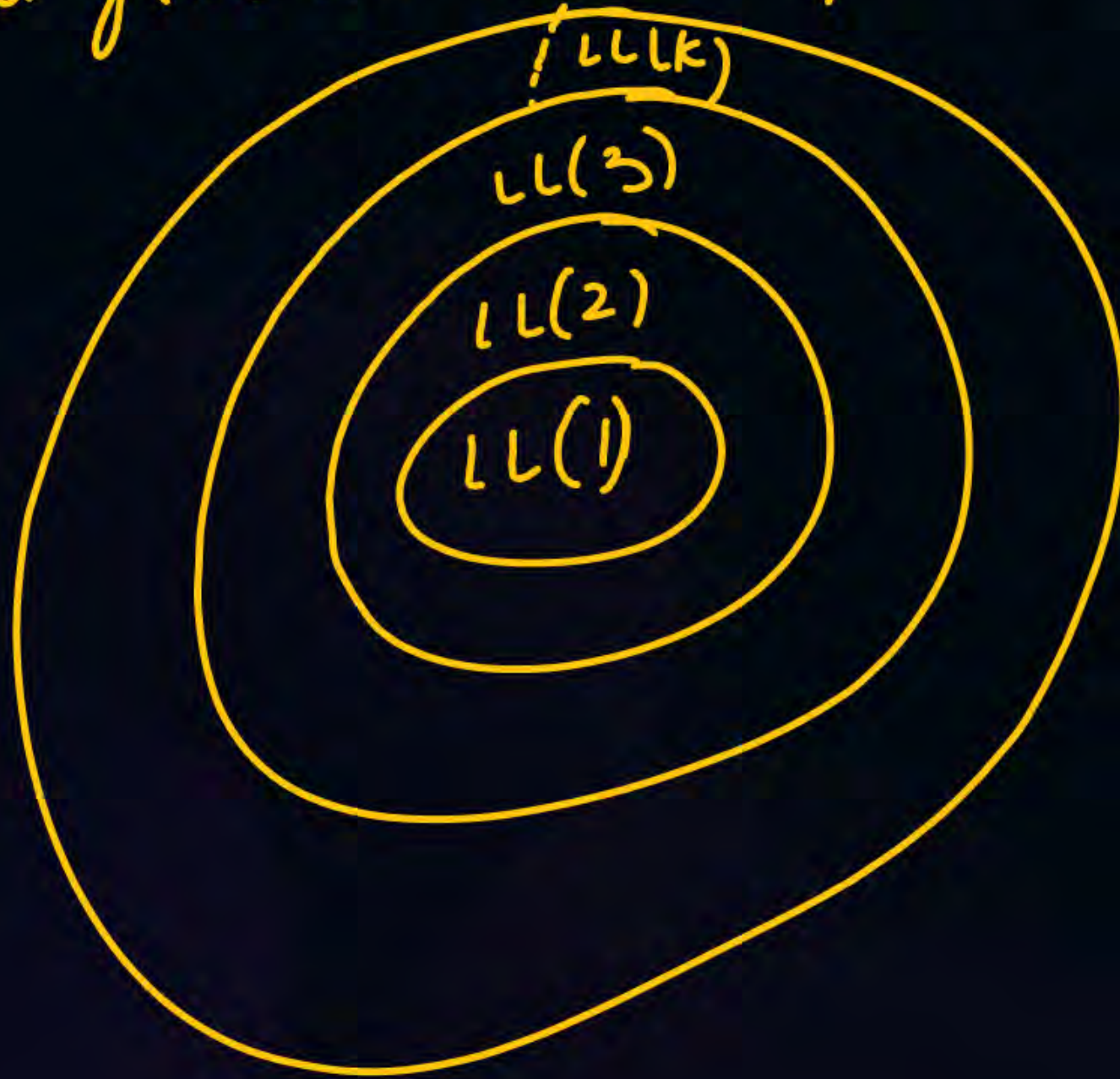
$LR(k)$



Bottom up



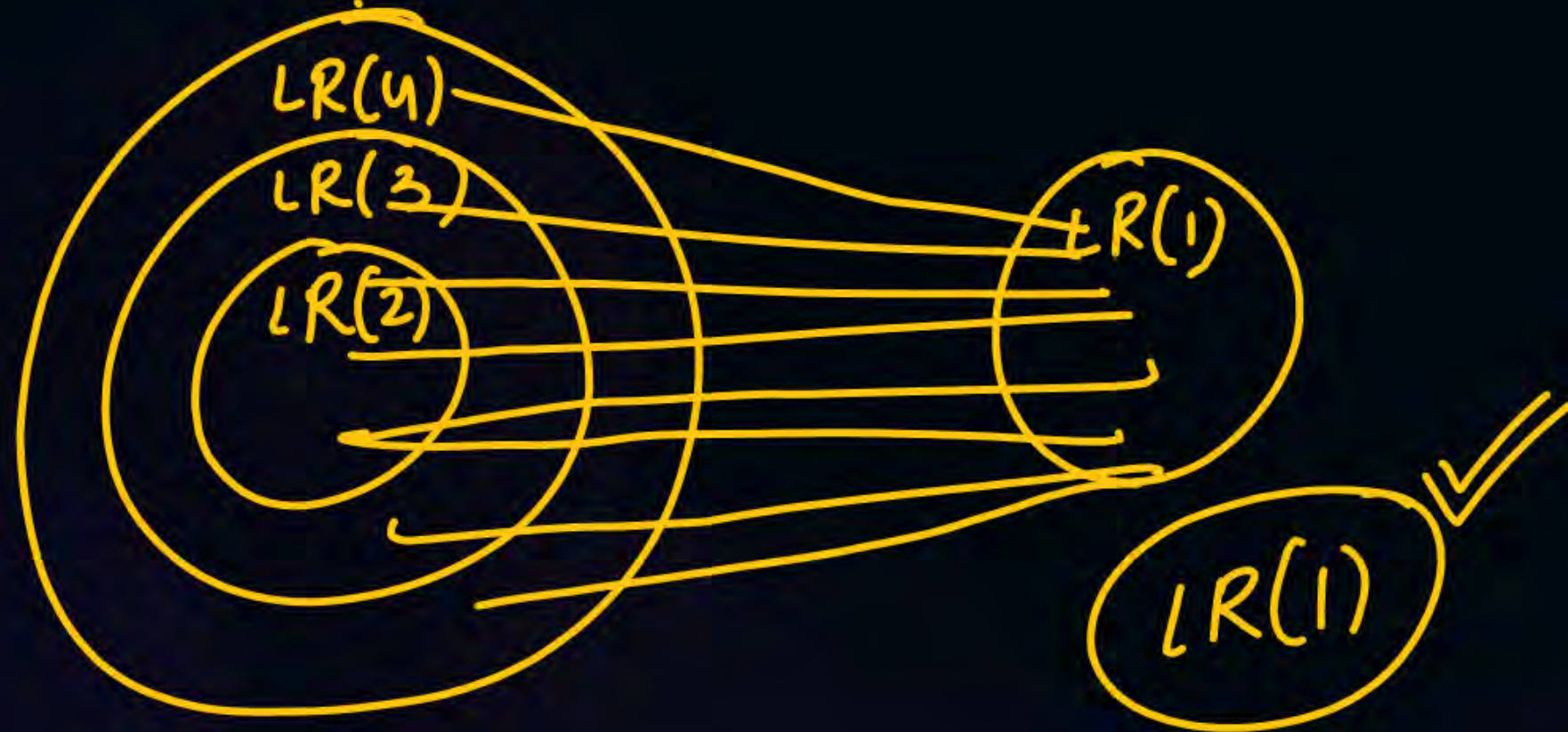
→ If a grammar is $LL(k)$ then it is surely $LL(k+1)$



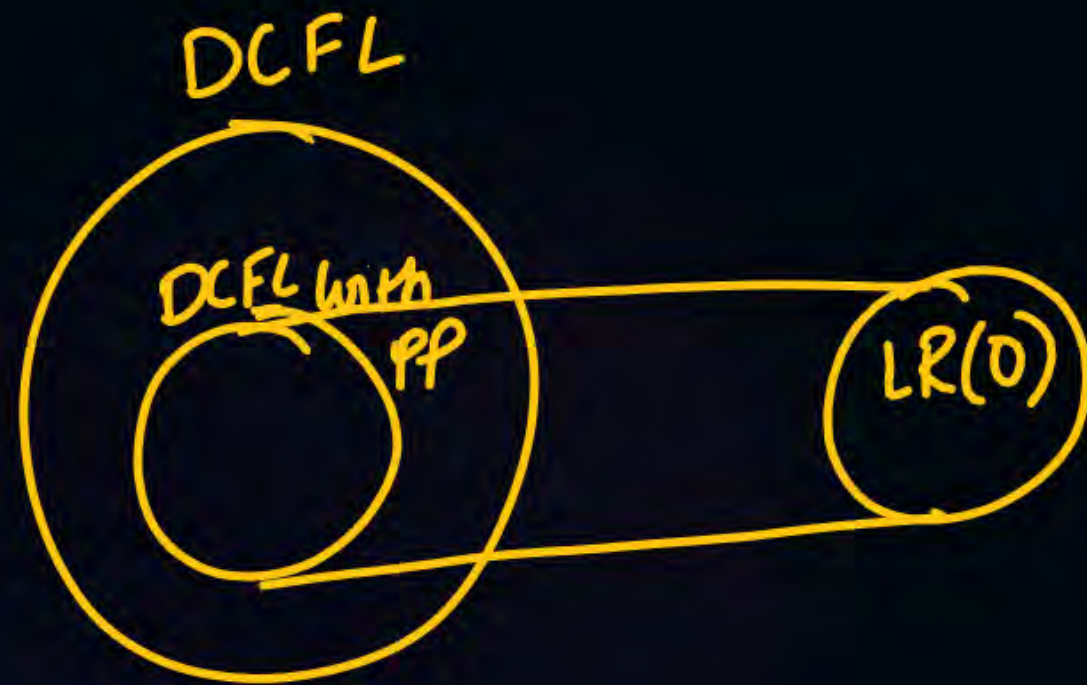
If a grammar is $LR(k)$ then it is surely $LR(k+1)$



- $\exists$ an algorithm to check if a grammar is $LL(k) \rightarrow$ Decidable
- " " " " " " $LR(k) \rightarrow$ Decidable
- If 'G' is $LR(k) / k \geq 2$, then $\exists G_1$ which is $LR(1)$ for the same language



→ $\forall$ DCFL with prefix property, we can generate LR(0) parser



What is prefix property?

DCFL 'L' has prefix property if $\forall w \in L$, there is no proper prefix $(w) \in L$

Ex: $L = \{011, 10\}$

$\epsilon, 0, 01$ $\epsilon, 1$

L has prefix property. So LR(0) can be generated

$a^n b^n / n \geq 1 \rightarrow$ Prefix property $\rightarrow LR(0)$ can be generated

aaabbb

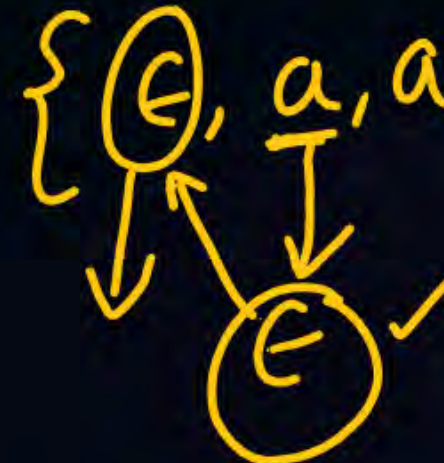
$\swarrow$
 $\epsilon, a, aa, aaa, aaab, aaabb$
 $\times \quad \times \quad \times \quad \times \quad \times \quad \times$

$L = \{011, 10, 01\} \rightarrow$ no prefix property
 $\rightarrow$ no LR(0) parser

$\epsilon, 0, 01$ $\epsilon, 01,$

$$L = a^*b^*$$

$\{\epsilon, a, aa, aaa, \dots, b, bb, bbb, \dots, ab, aab, \dots\}$



Whenever ϵ is in the language, it has no prefix property
 $LR(0)$ is not possible

$$a^n b^n / n \geq 0$$

$\in \rightarrow$ no prefix property

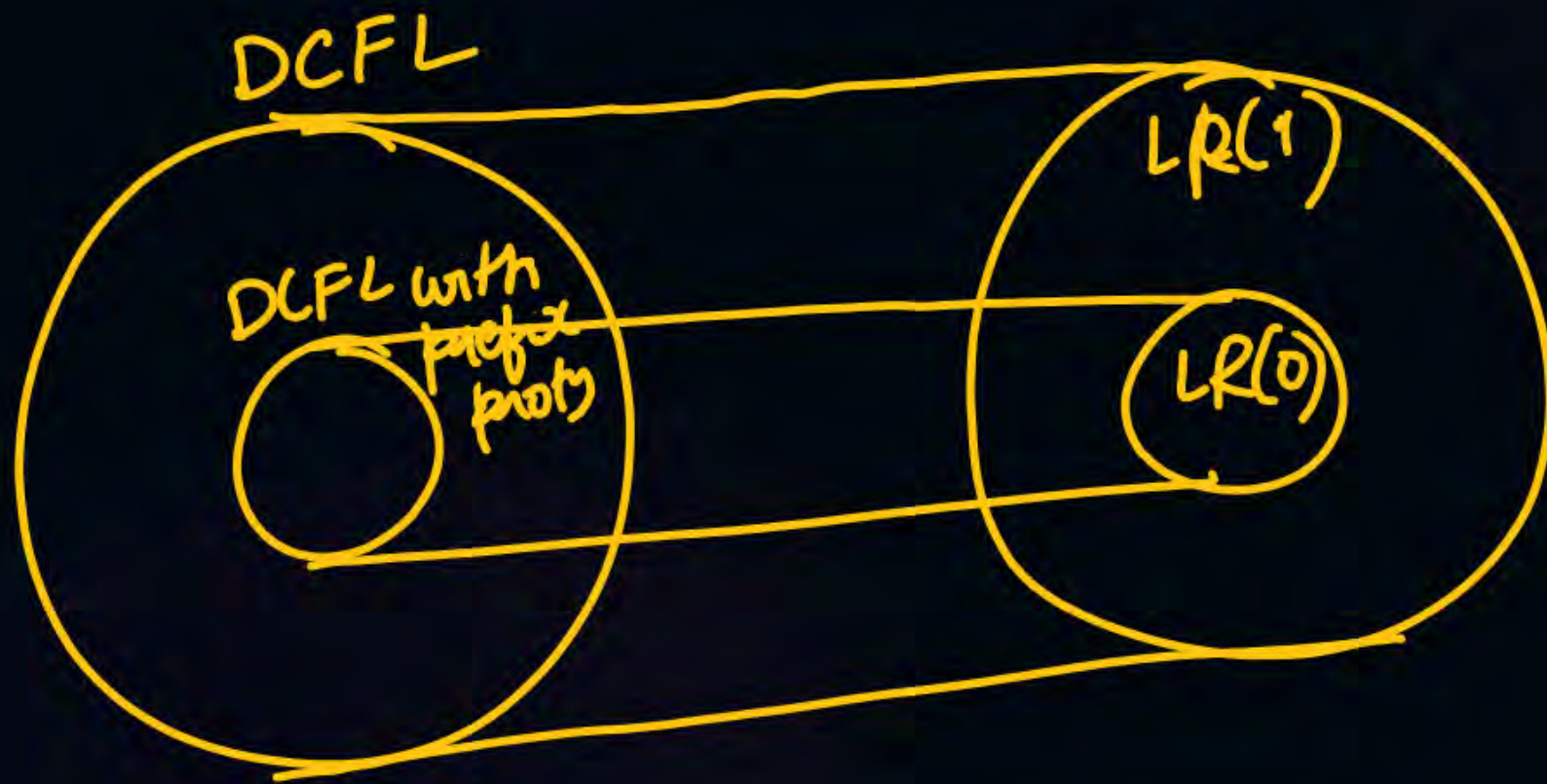
$\rightarrow$ NO LR(0) parser

$\underline{a^+ b^*} \rightarrow \underline{Reg} \rightarrow \underline{DCLFL} \rightarrow \underline{LR(1)} \checkmark$

$\{ \epsilon, a, aa, aaa, \dots, ab, abb, \dots \}$

ϵ, a

no prefix property
NO LR(0)



$$A \rightarrow A\alpha_1 / A\alpha_2 / \dots A\alpha_n / \beta_1 / \beta_2 / \dots \beta_n$$

$$A \rightarrow \beta_1 A' / \beta_2 A' / \beta_3 A' / \dots \beta_n A'$$

$$A' \rightarrow \alpha_1 A' / \alpha_2 A' / \dots \alpha_n A' / \epsilon$$

$$A' \rightarrow \underline{A}\alpha / \beta$$



$$\underline{\underline{\beta(\alpha)^*}}$$

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' / \epsilon$$

CP

THANK - YOU